Marking Memo - mock-eisa-software_developer-paper-02

MOCK / PRACTICE ASSESSMENT ONLY. This paper is generated for formative self-assessment and exam-preparation purposes. It is NOT an official QCTO External Integrated Summative Assessment (EISA), is not moderated by MICT SETA, and confers no qualification status or credit.

- **Total Marks:** 100

Section A: Programming Fundamentals and Logical Reasoning

Question A1 (15 marks)

(1) - 3 marks

Model answer: 1101(2) = 1x2^3 + 1x2^2 + 0x2^1 + 1x2^0 = 8 + 4 + 0 + 1 = 13

Criteria:

- Correct positional/place-value setup shown (1 mark(s))
- Correct arithmetic at each place value (1 mark(s))
- Correct final decimal value (1 mark(s))

Accepted alternatives:

- Doubling method (start from leftmost bit, double-and-add) reaching the same final value.

(2) - 4 marks

Model answer: 17 % 5 = 2; 2 * 3 = 6; 2 + 6 = 8; 8 - 1 = 7. total = 7.

Criteria:

- Modulus evaluated correctly first (correct precedence) (1 mark(s))
- Multiplication evaluated before remaining addition/subtraction (1 mark(s))
- Left-to-right evaluation of remaining +/- operators (1 mark(s))
- Correct final value stated (1 mark(s))

(3) - 4 marks

Model answer: n=0: 0%2==0 -> print 0; n=1: skip; n=2: print 4; n=3: skip; n=4: print 8; loop ends at n=5. Output: 0, 4, 8.

Criteria:

- Correct loop boundary (correct number of iterations) (1 mark(s))
- Correct condition evaluation at each iteration (2 mark(s))
- Correct final output sequence/value stated (1 mark(s))

Accepted alternatives:

- Answer presented as a trace table instead of prose - equally acceptable.

(4) - 4 marks

Model answer: Python raises this TypeError because `+` between a `str` and a `float` is not defined - Python does not implicitly convert numbers to strings the way some other languages (e.g. JavaScript) do. Fix: `message = "Total due: " + str(45.50) + " ZAR"` (or `f"Total due: {45.50} ZAR"`), both of which produce `"Total due: 45.5 ZAR"`.

Criteria:

- Correctly explains that Python's `+` operator does not implicitly convert a float to a string (unlike some other languages) (2 mark(s))
- Provides a corrected line that runs successfully and produces the exact target string (2 mark(s))

Accepted alternatives:

- str(), f-strings, .format(), or % formatting are all acceptable ways to perform the conversion.

Section B: Front-End Web Development (HTML5, CSS, JavaScript, OOP)

Question B1 (30 marks)

(1) - 6 marks

Model answer: The task's stated requirement is that at least one interest-area checkbox MUST be checked before the form submits. A <form> containing <input type="text" name="fullName" required>, <input type="email" name="email" required>, and one <input type="checkbox"> per interest area, each paired with a <label>, satisfies the markup part of this. However, HTML5's `required` attribute only ever validates a single element's own value - the browser has no built-in concept of 'at least one checked' across a group of checkboxes, so this specific requirement CANNOT be enforced by HTML5 attributes alone. Enforcing it requires JavaScript (e.g. checking on submit that at least one checkbox in the group is checked) or a workaround such as a single hidden required proxy input tied to the group's state.

Criteria:

- Uses a <form> with semantic, appropriately-labelled <input> elements (2 mark(s))
- Uses type="email" and the required attribute correctly on the name/email fields (2 mark(s))
- Checkboxes correctly implemented and labelled for the interest areas (1 mark(s))
- Correctly states that HTML5 cannot natively enforce the stated 'at least one checkbox selected' requirement across a group, and correctly identifies that JavaScript (or an equivalent workaround) is needed to enforce it (1 mark(s))

Accepted alternatives:

- Use of <label for=...> or wrapping <label> - either is acceptable.
- Mentioning aria-required/role=group as an accessibility improvement is a bonus, not a substitute for the correct answer that native HTML5 validation cannot enforce this rule.

(2) - 6 marks

Model answer: `.form-field { display: flex; flex-direction: column; }` by default, then `@media (min-width: 768px) { .form-field { flex-direction: row; align-items: center; } }`. Flexbox suits this because it lets the same markup reflow between a stacked and inline layout by changing a single property (flex-direction) rather than rewriting positioning rules.

Criteria:

- Correct default (mobile-first) stacked flex layout (2 mark(s))
- Correct media query breakpoint and row layout for wider screens (3 mark(s))
- Sensible one-sentence justification for choosing Flexbox (1 mark(s))

Accepted alternatives:

- Any reasonable breakpoint value (e.g. 700px-800px) with correct reasoning is acceptable; 768px is the suggested convention, not a strict requirement.
- CSS Grid used instead of Flexbox, provided the same responsive behaviour is achieved and justified - award full marks for the layout, partial for the Flexbox-specific justification.

(3) - 8 marks

Model answer: ```javascript

form.addEventListener('submit', function(e) {

e.preventDefault();

const email = form.email.value;

const valid = email.includes('@') && email.indexOf('.', email.indexOf('@')) > -1;

if (!valid) {

errorEl.textContent = 'Please enter a valid email address.';

} else {

errorEl.textContent = '';

console.log(Object.fromEntries(new FormData(form)));

}

});

```

Criteria:

- Correctly attaches a submit event listener and calls preventDefault() (2 mark(s))
- Validates the email field per the stated rule (contains '@' and a '.' after it) (3 mark(s))
- Displays an inline error without alert() when invalid (2 mark(s))
- Logs form data to the console when valid (1 mark(s))

Accepted alternatives:

- Use of a regular expression for email validation instead of includes()/indexOf(), provided it correctly rejects/accepts the same cases.
- Reading field values via `document.getElementById` instead of the form's named-element access - equally acceptable.

**(4) - 10 marks**

*Model answer:* ```javascript

class Volunteer {

constructor(name, hoursPledged) {

this.name = name;

this._hoursPledged = hoursPledged;

}

get hoursPledged() {

return this._hoursPledged;

}
```

```
set hoursPledged(value) {
if (value < 0) throw new Error('hoursPledged cannot be negative');
this._hoursPledged = value;
}
}
```

This demonstrates ENCAPSULATION: the internal field is hidden behind an underscore-prefixed (or truly private #-prefixed) property, and all reads/writes are mediated by the getter/setter, which is what lets the class enforce the 'no negative values' invariant in one place.

Criteria:

- Correct ES6 class syntax with constructor (3 mark(s))
- Correct getter implementation (2 mark(s))
- Setter correctly rejects/handles negative values (3 mark(s))
- Correctly names encapsulation and gives a valid reason it matters here (2 mark(s))

Accepted alternatives:

- Use of true private fields (`#hoursPledged`) instead of the underscore convention.
- Setter that clamps to 0 or ignores the invalid assignment instead of throwing, provided the negative value never becomes the stored value.
- Candidate names a different but defensibly-linked principle (e.g. 'data hiding') if their justification correctly describes encapsulation's mechanism.

Section C: Systems Modelling with UML

Question C1 (15 marks)

(1) - 8 marks

Model answer: Member(memberId, name, contactEmail; reserveBook(), cancelReservation()); Book(bookId, title, isbn, status; isAvailable(), markOnLoan()); Reservation(reservationId, reservationDate, status; notifyMember(), expire()). Associations: Member 1..* Reservation (a member may have many reservations), Reservation *..1 Book (each reservation is for exactly one book, a book may have many queued reservations)

Criteria:

- At least 3 plausible attributes per class (all three classes) (3 mark(s))
- At least 2 plausible methods per class (all three classes) (2 mark(s))
- Correct associations identified between the classes (2 mark(s))
- Correct/plausible multiplicities stated (e.g. one-to-many) (1 mark(s))

Accepted alternatives:

- Any reasonably named attributes/methods that a competent developer would recognise as relevant - exact names are not required, only correct concepts and correct multiplicity direction.

(2) - 7 marks

Model answer: 1) Member requests reserveBook(bookId) on ReservationService. 2) ReservationService checks Book.status. 3) Book is on loan, so ReservationService creates a Reservation and adds Member to

the waitlist. 4) When the Book is returned, ReservationService updates Book.status and calls notifyMember() on the next waitlisted Reservation.

Criteria:

- Correct initiating actor and first message (2 mark(s))
- Correctly identifies the 'currently unavailable' branch (on loan / slot full) and the waitlist step (3 mark(s))
- Includes a service/coordinator object mediating between the actor and the resource (1 mark(s))
- Correct final notification step once availability changes (1 mark(s))

Accepted alternatives:

- Any logically consistent object names, provided the sequence of decisions and the waitlist/notify mechanism is correct.

Section D: Data, Databases and Querying

Question D1 (20 marks)

(1) - 6 marks

Model answer: CREATE TABLE Suppliers (supplier_id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, contact_email VARCHAR(100));

CREATE TABLE Products (product_id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, quantity_in_stock INT NOT NULL, reorder_threshold INT NOT NULL, supplier_id INT, FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id));

Criteria:

- Both tables created with sensible, correctly-typed columns (2 mark(s))
- Primary key correctly defined on each table (2 mark(s))
- Foreign key correctly defined linking the two tables (2 mark(s))

Accepted alternatives:

- Any reasonable equivalent data types for the target SQL dialect (e.g. SERIAL vs INT for the PK, TEXT vs VARCHAR).
- AUTO_INCREMENT/IDENTITY on the primary key is a valid addition, not a requirement.

(2) - 6 marks

Model answer: SELECT p.name, p.quantity_in_stock, s.contact_email FROM Products p JOIN Suppliers s ON p.supplier_id = s.supplier_id WHERE p.quantity_in_stock < p.reorder_threshold ORDER BY p.quantity_in_stock ASC;

Criteria:

- Correct JOIN between the two tables on the foreign key (2 mark(s))
- Correct WHERE condition selecting below-threshold rows (2 mark(s))
- Correct ORDER BY direction and column (2 mark(s))

Accepted alternatives:

- Implicit join syntax (WHERE p.supplier_id = s.supplier_id) instead of explicit JOIN - equally acceptable.
- Column order in the SELECT list may differ from the model answer.

(3) - 4 marks

Model answer: UPDATE Products SET price = price * 1.10 WHERE supplier_id = 3; Omitting the WHERE clause would apply the price increase (or reset) to every row in the table, silently corrupting data for rows that were never meant to change.

Criteria:

- Syntactically correct UPDATE statement with the correct filter (2 mark(s))
- Correct explanation of the risk of omitting WHERE (mass/unintended update) (2 mark(s))

(4) - 4 marks

Model answer: Use optimistic concurrency control: include a version number or last-updated timestamp column, and require the UPDATE's WHERE clause to match the version the user last read; if no row matches (because someone else updated it first), reject the save and ask the user to reload and retry. (Pessimistic row-locking during the edit session is an equally valid alternative.)

Criteria:

- Names a valid concurrency-control technique (optimistic locking, pessimistic locking/row locks, or transactions with appropriate isolation level) (3 mark(s))
- Explains, even briefly, how it prevents a lost update (1 mark(s))

Accepted alternatives:

- Database transactions with SERIALIZABLE/REPEATABLE READ isolation.
- Application-level 'last write wins with warning' design, provided the candidate explains it actually surfaces the conflict rather than silently discarding data.

Section E: SDLC, Algorithms and Secure Coding

Question E1 (15 marks)

(1) - 4 marks

Model answer: Testing - the update describes running the completed build past real users specifically to find defects/issues before a wider release, which is the defining activity of the testing phase (this also has UAT characteristics, which is acceptable to name).

Criteria:

- Names the correct SDLC phase (2 mark(s))
- Justification correctly ties the phase name to the specific activity described (2 mark(s))

Accepted alternatives:

- 'User Acceptance Testing' as a more specific answer for the feedback scenario, since it is a sub-activity of testing.

(2) - 6 marks

Model answer: ```

function findDuplicate(ids):

seen = empty set

for id in ids:

if id in seen:

return id

seen.add(id)

return None

```

Applied to a list of customer IDs, this returns the first ID/number already seen. Time complexity: O(n), because each element is visited once and set membership checks/inserts are O(1) on average.

Criteria:

- Correct algorithm logic that reliably detects a duplicate (3 mark(s))
- Uses a set/hash-based lookup rather than a nested loop (or explicitly notes the nested-loop alternative's worse complexity) (1 mark(s))
- States O(n) (or correctly justifies whatever complexity their approach actually has, e.g. O(n^2) for a nested-loop version) (2 mark(s))

Accepted alternatives:

- A correct nested-loop (O(n^2)) solution, provided the candidate correctly states O(n^2) rather than incorrectly claiming O(n) - full marks require complexity/approach consistency, not necessarily the optimal algorithm.
- Sorting the list first then scanning for adjacent duplicates (O(n log n)), correctly justified.

**(3) - 5 marks**

*Model answer:* This is vulnerable to SQL injection: user_input is concatenated directly into the query string, so input like `'; DROP TABLE feedback; --` would execute as SQL. Fix with a parameterised query, e.g.:

```python

query = "SELECT * FROM feedback WHERE customer_name = %s"

cursor.execute(query, (user_input,))

```

This is safe because the database driver sends the value separately from the query structure, so it can never be interpreted as SQL code.

Criteria:

- Correctly names SQL injection as the vulnerability (2 mark(s))
- Rewrites using a parameterised query / prepared statement (or equivalent ORM safe-query method) (2 mark(s))
- Explains, even briefly, why the fix is safe (1 mark(s))

Accepted alternatives:

- Use of an ORM's query builder (e.g. Django ORM, SQLAlchemy filter) instead of raw parameterised SQL.
- Placeholder syntax specific to the candidate's chosen language/driver (?, %s, $1, :name) - any correct parameterisation mechanism is acceptable.

Section F: Workplace Integration and Professional Practice

Question F1 (5 marks)
```

Model answer: The relevant principle is data protection / privacy legislation (e.g. POPIA-style personal information protection) and workplace data-governance policy: a customer contact list is personal information, and 'not confidential' is not the same as 'lawful to share' - export and use of personal data must be limited to what the client actually needs and agreed to. Before complying, the developer should check the data processing agreement/scope with the client, confirm whether raw contact details are actually required (versus just order/product data), and escalate to a supervisor or data-governance contact if the request appears to exceed the agreed purpose, rather than silently complying or silently refusing.

Criteria:

- Names a specific, correctly-applicable governance/ethical/legislative or professional-practice principle (not a vague 'be ethical') (2 mark(s))
- Explains a concrete, appropriate action to take before complying (e.g. check scope, escalate, confirm necessity) rather than blanket compliance or blanket refusal (2 mark(s))
- Answer is coherent and specific to the scenario given, not generic boilerplate (1 mark(s))

Accepted alternatives:

- Any correctly-reasoned principle in the same family (e.g. 'confidentiality by design', 'least-privilege data sharing', 'informed consent') is acceptable even if named differently from the model answer.

Partial credit guidance: A candidate who identifies the right concern but proposes an overly extreme response (flat refusal with no escalation path, or unquestioning compliance) should lose the second criterion's marks but may still earn the first and third.